Cut Before: Find the Promos

Kuku AI Generalist Assignment – Problem 1

Atit Kumar Satsangi Roll: 230241 Civil Engineering

1. The problem and my approach

An editor has to read or watch every episode to find a short moment that makes a new viewer want more. I chose this problem because the supplied scripts let me check exact cut points. My idea is simple: **give the viewer enough to care, then cut before the answer.** The tool finds and ranks promo moments. It does not edit video or add music, dialogue or text overlays.

I use one continuous 20–60 second segment per selected episode. This keeps the first version small and easy to check; it does not search for montages or multiple promos within one episode. I treat the supplied timestamps as episode-relative script times. A quiet episode can be rejected. Actual footage still needs an editor's check.

2. How the tool works

Read and suggest. Python reads a timestamped TXT screenplay or a ZIP of episode TXT files. It records an input fingerprint, reads every timed event and finds valid 20–60 second windows. Simple story signals produce up to three varied suggestions per episode. These are starting points, not final decisions.

Ask the model. Each review packet contains the episode's full timeline, rule suggestions and a fixed JSON response format. The model judges the first three seconds, the unresolved ending, clarity for a new viewer and the stakes. It can change the boundaries, flag a concern or reject the episode. The new-input API route makes one generation request per episode; it does not load a fixed story or saved answer.

Validate and rank. Code checks the input fingerprint, episode coverage, exact event boundaries, duration and whether the cut interrupts dialogue. Invalid answers stop the run rather than becoming invented results. Valid output becomes an HTML report, CSV and JSON. Each retained promo includes its episode, ordered segment start/end times, hook, cliffhanger and why it works. Source packets, raw API responses and validation records are saved separately.

3. Results from the complete input pack

The scan read **50 episodes and 3,727 timed events**, checked 15,152 valid windows and produced 149 rule suggestions. The saved AI review covers all 50 episodes: **17 test first, 20 review further and 13 not recommended**. All 37 retained clips pass the duration, exact-boundary and uncut-dialogue checks.

Result provenance: these submitted decisions were made through AI chat review in Codex, then validated and exported by Python. They are not API-generated full-pack results. A separate live Gemini run on episode 44 subsequently passed validation. The API adapter is the separate route for fresh inputs and stronger interview models. Both routes use the same review format and validator.

4. What I would post first, and why

1. Episode 12: 00:44.00–01:09.20 (25.2 s). A sister demands that Juliana crawl into thorns. The clip ends with a threat to hang her husband. The cruel choice is easy to understand and leaves a strong unanswered question. This is my first test.

2. Episode 23: 01:19.00–02:04.00 (45.0 s). "Someone is poisoning her" gives an immediate mystery. Someone approaches the room at the end. I would test this next, while checking whether the preparation in the middle feels slow.

3. Episode 18: 00:39.50–01:08.00 (28.5 s). They cannot escape a fire, yet she will not leave a book. The ceiling warning ends the clip before the outcome. This offers visual danger after the two dialogue-led ideas.

5. Ranking and a mistake we found

The model rates hook, cliffhanger, standalone clarity and stakes from 0 to 5. The weights are 30%, 35%, 25% and 10%. "Test first" ranks ahead of "review further"; within each group the weighted score orders the clips. These are editorial judgments, not predicted clicks or recruitment decisions.

The rule scorer ranked episode 11's **00:16.80–00:37.10** cut second overall. It opens with a theft accusation but ends on another accusation: "He threatens me again!" That repeats the conflict instead of creating a new question. The review moved the end to **00:53.60**, when Alexander allows the search. The viewer now wants to know what will be found.

In a self-audit of the baseline's top ten suggestions, the same AI kept 2 unchanged, flagged 5 and rejected 3. This identifies failures; it is not an independent accuracy score. The original baseline, saved review and audit are preserved so reviewers can see the changes.

6. Rerun with a new story and a stronger API model

Open the linked Colab notebook and run setup. Upload a fresh timestamped TXT or ZIP. In the settings, choose **Gemini**, **OpenAI** or **Other compatible API**; enter the supplied model ID. For a custom provider, also enter its documented base URL, Responses or Chat Completions protocol, and token field. Native Gemini uses generateContent. Native APIs with different formats need another adapter.

First tick `RUN_API` and `TEST_CONNECTION_ONLY`. Enter the key in the hidden prompt. This sends one small JSON generation request without the screenplay; model listing alone does not prove generation access. After a successful check, untick the test option, rerun the settings cell and run the review cell. It reviews every uploaded episode. Download the resulting ZIP. Increasing model strength requires changing settings, not the ranking or validation code. The output token budget is also configurable.

Keys are sent in the provider's authentication header, never placed in the URL or saved in output files. The notebook clears its key variable after the attempt. Errors distinguish common key, permission, model, parameter and quota problems without printing private response bodies. There are no automatic retries or silent model replacements.

7. What has actually been tested

Passed: 29 automated tests covering parsing, timing, review validation, authentication headers, URL handling, error privacy and simulated API responses. The actual notebook cells were also run locally with simulated fresh TXT/ZIP uploads for all three API formats, including validation and ZIP download. Hosted Colab setup, a real screenplay upload and Gemini model listing worked.

Live generation passed: in hosted Colab, `gemini-3.5-flash` passed the small connection test, then reviewed the uploaded episode 44. It produced "Chosen in Chains, **00:46.40–01:10.50 (24.1 s)**, and the validator accepted one episode decision and one clip. The earlier `gemini-2.5-flash` attempt returned HTTP 404. This confirms the Gemini workflow on one real episode, not API generation of the full 50-episode pack or live testing of every supported provider.

8. Scale, evaluation and tools used

At larger scale, I would cache unchanged episodes by fingerprint, use bounded parallel API requests, track tokens and failures, and let editors check the strongest clips. These are next steps, not implemented scale claims. I would ask two editors to review 30 suggestions, measure usable clips and time saved, then compare audience retention under the same test conditions. No audience uplift has been measured; script timing alone cannot prove pacing, picture cuts or a strong hook.

I used **Codex** for code, story judgments, debugging and writing; **Python** for parsing and checks; and **LaTeX** for this document (the earlier draft used ReportLab). The episode 11 mistake shows why AI review needs checking. The free chat route remains documented, as the brief permits a smaller free version. Source code, the notebook and full saved outputs are delivered; confidential excerpts are shared only through the complete protected review link in the submission form.

Prototype and notebook: `https://github.com/atits23/cut-before-kuku-assignment`